

The Filtering Approaches for the Improved Boyer-Moore Approximate String Matching

Yi Kung Shieh and R. C. T. Lee

Department of Computer Science

National Tsing Hua University

d9762814@oz.nthu.edu.tw and rctlee@rctlee.cyberhood.net.tw

Abstract

The Boyer-Moore algorithm is to solve exact string matching. Here, the Bad Character Rule of the Boyer-Moore algorithm is extended to solve approximate string matching. Although Tarhio and Ukkonen introduce a basic algorithm, it is similar to the Horspool algorithm. We utilize the concept of their algorithm to implement the Bad Character Rule, and we will obtain a new shift length. When the window needs to be shifted in filtering stage, there is a chance to shift larger. This paper also explains two simple filtering approaches, and we easily combine any one of the filtering method to our algorithm. These filtering rules are easy to understand. One of them comes from the obvious concept of the definition of edit distance. Another uses a special relationship between edit distance and Hamming distance.

and the rest of the locations are verified by the U algorithm. This filtering approach has good performance in low error ratio $\alpha=k/m$ and large alphabet size. This paper introduces two filtering approaches. These two filtering approach can be easily combined with the TU algorithm. We also explain a new shift method of shifting the window for the TU algorithm. By using our algorithm, we obtain a new shift length. When we need to shift the window, we choose the larger one of the TU algorithm and our algorithm to be the shifting length. Therefore, we have more chance to shift longer and decrease the number of character comparisons.

In Section 2, we explain the TU algorithm. In Section 3, we show a variant of the TU algorithm in checking phase. We utilize the filtering approach of the FN algorithm and introduce a new filtering rule. We also show an improved shift method of the TU algorithm. The experiments are shown in Section 4.

1 Introduction

Approximate string matching problem is an important part of classical string algorithms. The definition of the problem is: given a text string T with length n , a pattern string P with length m and an integer threshold k , find all positions in T which are the end locations of substrings whose edit distance to P is at most k . The edit distance is the minimum number of edit operations, which include insertions, deletions and substitutions, converted one string to the other.

The problem can be solved by using dynamic programming approach (the Seller algorithm) [10] in $O(mn)$. Ukkonen improved the Seller algorithm and the algorithm is called the U algorithm [12] whose expected running time is $O(kn)$. Although Landau and Vishkin [7] and Galil and Park [4] explained algorithms whose worst case running times both are $O(kn)$, the performance of the U algorithm is better than the two algorithms [2,6,9]. Tarhio and Ukkonen [10] showed a filtering algorithm, which is called the TU algorithm. The TU algorithm is a sliding window approach. It filters the part of impossible locations,

2 The TU algorithm

The TU algorithm (which is also called the BMH algorithm) [11] is a filtering algorithm. It contains two phases in searching: filtering phase and checking phase. The algorithm is shown in Figure 1.

2.1 The filtering phase

In filtering phase, it opens a window $W=T(i-m+1,i)$, where i is a location of T and $T(i-m+1,i)$ is the substring $t_{i-m+1}t_{i-m+2}\dots t_i$ in T . Initially, the window is at the leftmost side of T , i.e. $W=T(1,m)$. The algorithm compares the suffix $T(i-m+k+1,i)$ in W with the suffix $P(k+1,m)$ in P from right to left. If a character t_{i-j} , where $0 \leq j < m-k$, mismatches its corresponding character p_{m-j} of P , we cannot directly claim that an error exists at location $i-j$ of W . It is possible that after we apply some edit operations on P , like insertions or deletions, t_{i-j} might exactly match its corresponding character of P . Therefore, we have to find whether t_{i-j} exists in $p_{m-j-k}\dots p_{m-j+k}$ or not (the scope $p_{m-j-k}\dots p_{m-j+k}$ is called k -environment of p_{m-j} , denoted as $C(k,m-j)$, as shown in Figure 2.

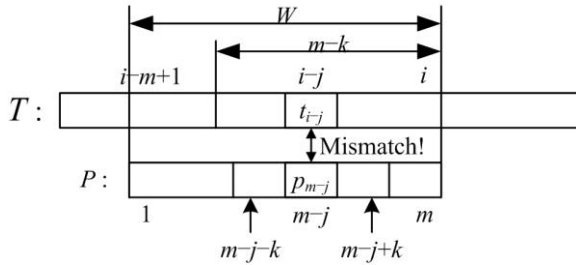
```

01. for  $i = 0$  to  $k$  do
02.   for  $\alpha$  in  $\Sigma$  do
03.      $d[i][\alpha] = m-i$ ;
04.   for  $i = 0$  to  $k$  do
05.     for  $j = 1$  to  $m$  do
06.        $d[i][p_j] = m-i-j$ ;
07.    $j = m$ ;
08.   while  $j \leq n$  do begin
09.      $r = j$ ;
10.      $i = m$ ;
11.      $s = m$ ;
12.      $num = 0$ ;
13.     while  $i > k$  and  $bad \leq k$  do begin
14.       if  $i \geq m-k$  then  $s = \min(s, d[m-i][t_r])$ ;
15.       if  $t_r$  does not exist in  $C(k, i)$  then  $num = num + 1$ ;
16.        $r = r - 1$ ;
17.        $i = i - 1$ ;
18.     end
19.     if  $num \leq k$  then
20.       mark the locations from  $j-m-k$  to  $j-m+k$ 
21.        $j = j + \max(k+1, s)$ ;
22.   end

```

Figure 1. The TU algorithm

If yes, the algorithm continues to compare the rest part of the suffix $T(i-m+k+1, i)$. Otherwise, we said that there is an error at location $i-j$. We use an integer num to count the number of such error. The character of the first error is bad_0 , the second error is bad_1 and so on. There are $bad_0, bad_1, \dots, bad_{num-1}$.

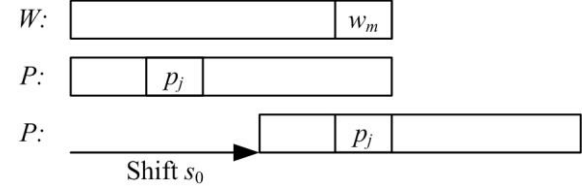
Figure 2. Finding whether t_{i-j} exists in $C(k, m-j)$.

2.2 The shift approach

When we find that num is larger than k , it means W does not contain any possible solution. We have to shift the window. We know that if the edit distance between a string $A = a_1a_2\dots a_m$ and a string $B = b_1b_2\dots b_m$ is at most k , denoted as $ed(A, B) \leq k$, consider any substring $A(i, i+k-1)$ and $B(i, i+k-1)$, there must exist at least one exact match $a_j = b_j$ for $i \leq j \leq i+k-1$.

We focus on the last $k+1$ characters in W . We consider the last character of W , which is w_m . When we want to align the same character in P with w_m , we need to find a character which is equal to w_m in $p_1p_2\dots p_{m-1}$. If there are many characters which are equal to w_m in $p_1p_2\dots p_{m-1}$, we have to choose the rightmost one (the largest location). Assume the

rightmost one is p_j , we shift the window $m-j$ steps and it will cause an exact match. If there is no character which is equal to w_m in $p_1p_2\dots p_{m-1}$, we shift the window m steps. For w_m , we call this shift length s_0 , which is a possible shift length. We consider the last second character of W , which is w_{m-1} . We also want to shift a character which is equal to w_{m-1} in $p_1p_2\dots p_{m-2}$. If the rightmost one is p_j , the possible shift is $s_1 = m-1-j$. If no character is equal to w_{m-1} in $p_1p_2\dots p_{m-2}$, the possible shift is $s_1 = m-1$. Figure 3 shows the possible shift s_0 that p_j is the rightmost one and $p_j = w_m$.

Figure 3. The possible shift s_0

We obtain $k+1$ possible shift length by the approach. Here, we choose the minimum values among $s_0, s_1, \dots, s_k$ to be the shift length s . We will explain how to mark the locations which are possible solutions. Because the marking approach, we can at least shift $k+1$ steps in order to avoid mark the part where we have marked before. Therefore, the shift length is the maximum value between s and $k+1$. We use the same method to shift the window until the whole text has been exhausted.

We construct the shift table d_r in pre-processing phase so that we can obtain the possible shift length s_r , for $0 \leq r \leq k$. Let the alphabet $\Sigma = \{\alpha_1, \alpha_2, \dots, \alpha_\sigma\}$ where σ is the size of Σ . Assume the rightmost α_u in $p_1p_2\dots p_{m-1-r}$ is at locations j of p , the value of $d_r[\alpha_u]$ is $m-r-j$, for $1 \leq u \leq \sigma$. For example, $P = tcgtag$ and $k = 1$. We construct d_0 and d_1 in pre-processing phase as shown in Table 1. $d_0[c] = 4$ means if we shift 4 steps, the rightmost c in $p_1\dots p_{m-1}$ align the t_i . $d_1[g] = 2$ means if we shift 2 steps, the rightmost g in $p_1\dots p_{m-2}$ align the t_{i-1} .

Table 1. The shift tables of $P = tcgtag$ and $k = 1$

	d_0	d_1
a	1	5
c	4	3
g	3	2
t	2	1

2.3 The marking and checking phase

If the whole suffix $T(i-m+k+1, i)$ of W has been compared and we only obtain $bad \leq k$, it means possible solutions exists in this part. We need to mark the locations which are the possible solutions, and we have to verify these marked locations in checking phase. We know that if location i is a solution, there

must be a substring S ended at location i in T such that $ed(S, P) \leq k$. The length of S must be between $m-k$ and $m+k$. So, the marked locations are the scope from $i-m-k$ to $i+m+k$. After the marking, we also need to shift the window. We use the above shift approach and go back to filtering phase.

We obtain the marking locations of whole T . We need to verify these locations. In the TU algorithm, it utilizes a dynamic programming table D with size $(m+1) \times (n+1)$. The boundary values are $D(i, 0) = 0$, for $0 \leq i \leq n$, and $D(0, j) = j$, for $0 \leq j \leq m$. The dynamic programming formula is:

$$D(i, j) = \begin{cases} D(i, j-1) + 1 \\ D(i-1, j) + 1 \\ D(i-1, j-1) + \text{if } t_i = p_j \text{ then } 0 \text{ else } 1 \end{cases}.$$

Let the diagonal d be $i-j$. We only obtain the values $D(i, j)$ where the location d are marked. When $D(i, j)$ refers an entry outside marked diagonals, the value of the entry can be taken to be ∞ . In the TU algorithm, it utilizes the Ukkonen's Cut-Off algorithm (the U algorithm) and obtains better performance.

3 The improved TU algorithm

3.1 Direct verification

When the TU algorithm finds the $bad \leq k$ in $W = T(i-m+1, i)$, it marks the locations from $i-m-k$ to $i+m+k$ which are possible solutions. After finding all possible solutions in T , we use a dynamic programming table to check these marked locations.

We can ignore the marking part. When we mark the locations from $i-m-k$ to $i+m+k$, it means in checking phase, we have to calculate the columns from $i-m-k$ to $i+k$ of dynamic programming table (because if any solution exists, it must be in the scope whose length is $m+2k$ as shown in Figure 4). In this situation, we directly use dynamic programming table with size $(m+1) \times (m+2k)$ to verify the locations from $i-m-k$ to $i+k$.

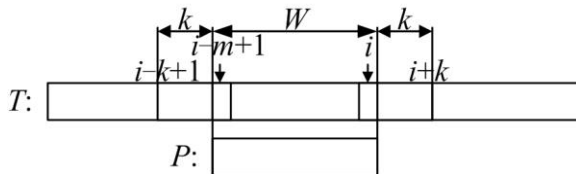


Figure 4. The scope $m+2k$ for possible solutions

The advantage of the direct verification is when the number of solutions is lesser, the performance is better.

3.2 The filtering approach of the FN algorithm

The FN algorithm is also solving the approximate string matching but it finds all substrings in T whose edit distance with P are at most k . The FN algorithm is a filtering algorithm, too. Before we explain the filter approach of the FN algorithm, we have to know the concept of nearest neighbor.

A and B are two strings. Let b' be a substring of B whose edit distance with A is the smallest among all substrings of B . Then b' is a nearest neighbor of A with respect to B and denoted as $NN(A, B)$. The edit distance between A and the nearest neighbor b' is called nearest neighbor distance, denoted as $NND(A, B)$. In other words, $NN(A, B) = b'$ and $NND(A, B) = ed(A, b')$.

The filtering approach of the FN algorithm uses a property as shown in Lemma 1.

Lemma 1. A and B are two strings and k is an error threshold. $A = a_1 + a_2 + \dots + a_r$. If $\sum_{i=1}^r NND(a_i, B) > k$, then $NND(A, B) > k$.

The proof is shown in [3].

By using Lemma1, it is easy to detect the edit distance between a substring of T and P is larger than k or not.

Now, we need an effective method to find the nearest neighbor in P for a substring of T . We explain a special relationship between edit distance and Hamming distance. We introduce Hamming distance first. Given two strings A and B with equal length, the Hamming distance between A and B , which is denoted $hd(A, B)$, is the number of positions for which the corresponding symbols are different. For example, $A = agctat$ and $B = acctcg$. It is easy to obtain that $hd(A, B) = 3$. The special relationship is shown in Lemma 2.

Lemma 2. Given two strings A and B with the same length 2, the Hamming distance and edit distance between A and B are equal.

Lemma2 can be easily proved by the method of exhaustion, we ignore the proof here. For example, $A = cg$ and $B = at$. $hd(A, B) = 2 = ed(A, B)$. $A = tc$ and $B = cc$, $hd(A, B) = 1 = ed(A, B)$.

According to Lemma2, we divide a substring S of T into many pieces with length 2 which are non-overlapping and continuous. For example, $S = acctcg$, and S is divided into $\{pi_1, pi_2, pi_3\} = \{s_1s_2, s_3s_4, s_5s_6\} = \{ac, ct, cg\}$, where pi_1 is the first

piece, pi_2 is the second piece and so on. We find the nearest neighbor distance for every piece. By Lemma1, we add the nearest neighbor distances of all pieces, if the summation is larger than k , it means $NND(S,P) > k$. Obviously, it implies that $ed(S,P) > k$.

We know $\Sigma = \{\alpha_1, \alpha_2, \dots, \alpha_\sigma\}$. There are σ^2 distinct permutations with length 2. We construct a table $NNDT$ with size σ^2 . It records the smallest Hamming distance of every permutation to substrings in P with length 2. For example, $P = actca$. $\Sigma = \{a, c, t\}$. The $NNDT$ is shown in Table 2. $NNDT[a][t] = 1$ means the $NND(at, P) = 1$. We can find that $NN(at, P) = ac$ or ct such that $hd(at, ac) = 1 = ed(at, ac)$ or $hd(at, ct) = 1 = ed(at, ct)$. It is easy to construct $NNDT$. We initialize the values in $NNDT$ to be 2 which means the nearest neighbor distance of every permutation is 2. We scan P from left to right. For every location i of P , we set $NNDT[p_i, \alpha_u] = \min(NNDT[p_i, \alpha_u], 1)$, $NNDT[\alpha_u, p_{i+1}] = \min(NNDT[\alpha_u, p_{i+1}], 1)$, and $NNDT[p_i, p_{i+1}] = 0$, where $1 \leq u \leq \sigma$.

Table 2. The table $NNDT$ of $P = actca$

		The 1 st character		
The 2 nd character	$NNDT$	a	c	t
	a	1	0	1
	c	0	1	0
	t	1	0	1

When we utilize the filtering approach of the FN algorithm to decide whether $W = T(i-m+1, i)$ can be filtered or not, we cannot use the whole W . We only consider the substring $S = T(i-m+1+k, i-k)$. If $NND(S, P)$ is larger than k , W can be ignored. The reason is if the edit distance between a string A and P is smaller than or equal to k , the length of A must be from $m-k$ to $m+k$. Assume that there is a substring S' in W and $S' = w_1 \dots w_{m-k}$ such that $ed(S', P) \leq k$. If we use the TU algorithm to compare W with P from right to left, W might be filtered because the unnecessary suffix w_{m-k+1} to w_m as show in Figure 5.

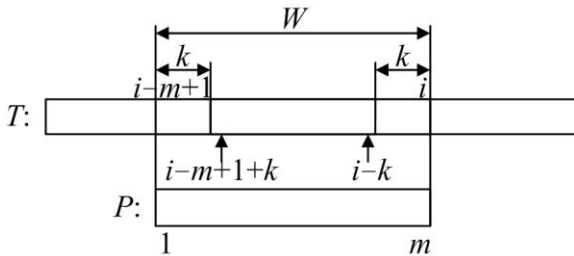


Figure 5. The filter scope of the FN's filter

So, when we add the filtering approach into the TU algorithm, we have to make sure the scope. We calculate the error every two characters. We cannot use num of the TU algorithm to count the number of

errors. We use $num2$ to count the errors by applying the filtering approach of the FN algorithm. The lines from 13 to 20 in Figure 1 can be modified as shown in Figure 6.

```

num2 = 0;
count = 1;
while i > k and num ≤ k and num2 ≤ k do begin
  if i < m-k then
    if count = 1 then temp=text[r]; count=0;
    else num2=num2+NNDT[temp][text[r]]; count=1;
  if i ≥ m-k then s = min(s, d[m-i][t_r]);
  if t_r does not exist in C(k, i) then num=num+1;
  r = r-1;
  i = i-1;
end
if num ≤ k and num2 ≤ k then
  mark the locations from j-m-k to j-m+k

```

Figure 6. Utilizing FN's filter in the TU algorithm

3.3 The Counting filtering approach

The filtering approach of the FN algorithm may not effective enough when the alphabet size is small and k is large. We introduce a simple filtering approach which is counting the number of every alphabet in P . This method is smaller influenced by the alphabet and k than the FN's.

The number of alphabet σ_u in $T(i-m+1+k, i)$ is x and in P is y , for $1 \leq u \leq \sigma$. If x is larger than y , we have to replace or insert $n_u = x - y$ characters in P for σ_u in $T(i-m+1+k, i)$. In this case, we obtain the errors between $T(i-m+1+k, i)$ and P will be at least n_u . Hence, if the summation $num3 = n_1 + n_2 + \dots + n_\sigma$ is larger than k , W can be ignored. Because the edit distance between the suffix $T(i-m+1+k, i)$ of W and P is larger than k , it is impossible that any possible solution exists in W .

We use a table $com1$ to record the number of every alphabet in P . $com1[\alpha_u]$ is number of α_u in P , for $1 \leq u \leq \sigma$. This step can be done in pre-processing. Before every comparing W with P , we set $num3$ to be zero and copy $com1$ to a new table $com2$. Consider w_j , for $m-k \leq j \leq m$, we check whether $com2[w_j]$ is zero or not. If yes, w_j is an error and $num3$ increases one. Otherwise, $com3[w_j]$ decreases one. When $num3 > k$, it means that we need more than k insertions or substitutions to transfer P to a substring ended at location i . Therefore, this window can be filtered. We can direct add this part by change the lines from 13 to 20 in Figure 1, as show in Figure 7.

3.4 A shift method for Bad Character Rule of Boyer-Moore algorithm

A possible shift length of the TU algorithm comes from aligning the rightmost character p_j in $p_1 \dots p_{m-r-1}$ with t_{i-r} such that $p_j = t_{i-r}$, for $0 \leq r \leq k$. It is

```

num3 = 0;
com2 = com1;
while i > k and num ≤ k and num3 ≤ k do begin
  if com2[tr] = 0 then num3 = num3 + 1;
  else com2[tr] = com2[tr] - 1;
  if i ≥ m - k then s = min(s, d[m - i][tr]);
  if tr does not exist in C(k, i) then num = num + 1;
  r = r - 1;
  i = i - 1;
end
if num ≤ k and num3 ≤ k then
  mark the locations from j - m - k to j - m + k
Figure 7. The counting filtering algorithm

```

obvious that we have not utilized the information of $bad_0, bad_1, \dots, bad_k$. By this information, we can apply the Bad Character Rule of Boyer-Moore algorithm [1] to be a new shift method. The Boyer-Moore algorithm is an exactly string matching algorithm. In an exactly string matching algorithm, when a mismatch occurs, we need to shift the window. The Bad Character Rule is when the first mismatch occurs ($w_j \neq p_j$), we need to shift a rightmost $w_{j'}$ in $w_1 \dots w_{j-1}$ to align with p_j such that $w_{j'} = p_j$. We can use the same method on the TU algorithm. We called this shift method bc_shift .

Assume $w_j \neq p_j$ and w_j is bad_0 , we try to find whether a character $w_{j'}$ exists in $p_1 \dots p_{j-1}$ and $w_{j'} = p_j$ or not. If yes, the possible shift s'_0 of bc_shift is $j - j'$. Otherwise, $s'_0 = j$. For example, $W = gcatca$, $P = tcgtag$ and $k = 1$. $bad_0 = w_5 = c$ and $bad_1 = w_3 = a$. For bad_0 , we find the rightmost $p_2 = c = w_5$ in $p_1 \dots p_4$. We obtain $s'_0 = 5 - 2 = 3$, as shown in Figure 8. For bad_1 , we cannot find $c = w_5$ in $p_1 \dots p_2$. We obtain $s'_1 = 3$. By the bc_shift method, we obtain the $k+1$ possible shift lengths. We choose the minimum shift among $s'_0, s'_1, \dots, s'_k$ be the shift length s' . If the window W can be filtered, we obtain the shift length is the maximum value of s , s' and $k+1$. The reason that we can choose the largest is all these three shift lengths are correct. Therefore, it won't be wrong if we choose anyone of these three values.

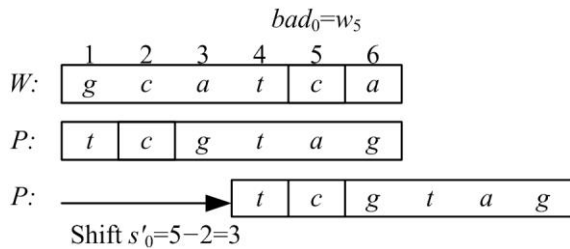


Figure 8. An example of bad_0 and s'_0

We must pay attention that if the window W cannot be filtered, we only use the shift length of the TU algorithm ($\max(s, k+1)$). When $num \leq k$, we do not know $bad_{num+1}, bad_{num+1}, \dots, bad_k$. Therefore, the

values of $s'_{num+1}, s'_{num+2}, \dots, s'_k$ are 1. In this situation, it is not useful. Hence, we use the original shift length.

We utilize the shift tables of the TU algorithm to implement bc_shift . First, we have to construct the tables from d_0 to d_{m-1} . If an error occurs at location j in W , we consult the table d_{m-j} . We will obtain a possible shift length $d_{m-j}[w_j]$.

For example, $W = gcatca$, $P = tcgtag$ and $k = 1$. First, we construct the tables from d_0 to $d_{m-1} = d_5$ as shown in Table 3. The $bad_0 = w_5$. We obtain $s'_0 = d_{6-5}[w_5] = d_1[c] = 3$. The $bad_1 = w_3$. We obtain $s'_1 = d_{6-3}[w_3] = d_3[a] = 3$.

Table 3. The shift tables d_0 to d_5 of an example

	d_0	d_1	d_2	d_3	d_4	d_5
a	1	5	4	3	2	1
c	4	3	2	1	2	1
g	3	2	1	3	2	1
t	2	1	3	2	1	1

It is easy to implement the bc_shift method in the TU algorithm. The Figure 9 shows the algorithm which combines the counting filtering approach and bc_shift method with the TU algorithm.

```

01. construct shift tables from  $d[0]$  to  $d[m-1]$ 
02. for  $a$  in  $\Sigma$  do
03.   com1[a] = 0
04. for  $i = 1$  to  $m$  do
05.   com1[pi] = com1[pi] + 1;
06. j = m;
07. while j ≤ n do begin
08.   r = j; i = m;
09.   s = m; s' = m;
10.   num = 0; num3 = 0;
11.   com2 = com1;
12.   while i > k and num ≤ k and num3 ≤ k do begin
13.     if com2[tr] = 0 then num3 = num3 + 1;
14.     else com2[tr] = com2[tr] - 1;
15.     if i ≥ m - k then s = min(s, d[m - i][tr]);
16.     if tr does not exist in C(k, i) then
17.       num = num + 1; s' = min(s', d[m - i][tr]);
18.     r = r - 1; i = i - 1;
19.   end
20.   if num ≤ k and num3 ≤ k then
21.     mark the locations from j - m - k to j - m + k
22.   if num > k then j = j + max(k + 1, s, s');
23.   else j = j + max(k + 1, s);
24. end

```

Figure 9. The improved TU algorithm

4 Experiments

Our machine is Intel Core 2 with 2.1GHz and 4GM RAM. We run Fedora 18 and all texts are on a local disk of 500GB. Our experiments were run on

texts of 10 MB and repeated 100 times (with different search patterns).

U is the U algorithm [12], TU is the TU algorithm [11], FN_TU is the TU algorithm with the FN's filtering approach [3], C_TU is the TU algorithm with the counting filter approach, BTU is the Bad Character Rule of Boyer-Moore algorithm [1] which we mention in Section 3.4, FN_BTU is the BTU with FN's filtering approach, and C_BTU is the BTU with counting filtering approach. The mark "(D)" means that the TU algorithm with non-mark method which we explain in Section 3.1.

Table 4 shows the results that the length of pattern is 10. The unit in this table is second. We can see that TU(D) is faster than other methods in the case of $\sigma=26$ and $k=1$. Because there were seldom solutions in this situation, TU(D) did not need to check the marking locations in dynamic programming table for the TU algorithm. In the case that $\sigma=4$ and $k=3$, the algorithms based upon the TU(D) algorithm were not well. FN_TU(D) had well performance because of the large alphabet size. The performance is proportional to the alphabet size of pattern. U always had good performance in the small error thresholds.

Table 4. Results for $m=10$ and varying k .

σ	$k = 1$		$k = 2$		$k = 3$	
	4	26	4	26	4	26
U	1.048	0.653	<u>1.463</u>	0.934	<u>1.880</u>	1.182
TU	0.671	0.120	3.326	0.200	5.166	0.482
FN_TU	0.672	0.128	3.062	0.212	5.528	0.550
C_TU	0.752	0.185	3.188	0.292	5.266	0.563
BTU	1.088	0.143	7.593	0.229	10.270	0.657
FN_BTU	0.821	0.170	6.354	0.225	10.030	0.624
C_BTU	0.753	0.191	3.155	0.310	5.296	0.590
TU(D)	0.663	<u>0.065</u>	4.242	0.146	6.077	<u>0.406</u>
FN_TU(D)	<u>0.584</u>	0.082	3.714	<u>0.143</u>	6.183	0.417
C_TU(D)	0.749	0.126	3.748	0.205	6.100	0.441
BTU(D)	0.667	0.083	4.204	0.174	6.048	0.452
FN_BTU(D)	0.594	0.107	3.645	0.177	6.390	0.470
C_BTU(D)	0.737	0.145	3.778	0.224	6.109	0.459

Table 5 shows the results that the length of pattern is 40. The unit in this table is also second.

Table 5. Results for $m=40$ and varying k .

σ	$k = 1$		$k = 2$		$k = 3$	
	4	26	4	26	4	26
U	1.099	0.635	1.490	0.973	2.036	1.273
TU	0.380	0.087	0.649	0.152	2.660	0.233
FN_TU	0.458	0.089	0.860	0.177	2.830	0.286
C_TU	0.505	0.127	0.828	0.212	1.991	0.332
BTU	0.363	0.100	0.718	0.162	11.590	0.241
FN_BTU	0.422	0.097	0.864	0.157	10.403	0.228
C_BTU	0.472	0.133	0.782	0.215	<u>1.938</u>	0.318
TU(D)	0.335	<u>0.038</u>	0.602	0.086	3.514	<u>0.163</u>
FN_TU(D)	0.393	<u>0.038</u>	0.775	<u>0.092</u>	3.837	0.174
C_TU(D)	0.485	0.061	0.851	0.133	2.469	0.227
BTU(D)	<u>0.323</u>	0.051	<u>0.575</u>	0.098	3.442	0.172
FN_BTU(D)	0.370	0.051	0.745	0.108	3.857	0.191
C_BTU(D)	0.439	0.067	0.798	0.138	2.422	0.216

The BTU and BTU(D) had well performances in long length of pattern. Because when σ and k is small and m is large, the TU algorithm always shifts slowly. But, BTU has another shift length can be chosen. Therefore, BTU may obtain larger shift.

5 Conclusion

Although our shift approach of Bad Character Rule applied to the TU algorithm may obtain better shift lengths, the performances are not very well in the above results. Therefore, we still have many details that can be improved.

Reference

- [1] Boyer, R. S. and Moore, J. S. 1977. A fast string searching algorithm. Communications of the ACM. Vol. 20, No. 10, pp.762-772.
- [2] Chang, W. I. and Lampe, J. 1992. Theoretical and Empirical Comparisons of Approximate String Matching Algorithms. In Proceedings of the Third Annual Symposium on Combinatorial Pattern Matching (CPM'92). Vol. 664, pp.172-181.
- [3] Fredriksson, K. and Navarro, G. 2004. Average-optimal single and multiple approximate string matching. Vol. 9, No. 1.4, pp. 1-47.

- [4] Galil, Z. and Park, K. 1990. An improved algorithm for approximate string matching. *SIAM Journal on Computing*. Vol. 19, No. 6, pp.989-999.
- [5] Horspool, R. N. 1980. Practical fast searching in strings. *Software - Practice and Experience*. Vol. 10, No. 6, pp.501-506.
- [6] Jokinen, P., Tarhio, J. and Ukkonen, E. 1996. A Comparison of Approximate String Matching Algorithms. *Software - Practice and Experience*. Vol. 26, No. 12, pp. 1439-1458.
- [7] Landau, G. and Vishkin, U. 1989. Fast parallel and serial approximate string matching. *Journal of Algorithms*. Vol. 10, No. 2, pp.157-169.
- [8] Levenshtein, V. I. 1966. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*. Vol. 10, No. 8, pp. 707-710.
- [9] Myers, G. 1999. A fast bit-vector algorithm for approximate string matching based on dynamic programming. *Journal of the ACM*. Vol. 46, No. 3, pp. 395-415.
- [10] Sellers, P. H. 1980. The theory and computation of evolutionary distances: pattern recognition. *Journal of Algorithms*. Vol. 1, No. 4, pp. 359-337.
- [11] Tarhio, J. and Ukkonen, E. 1993. Approximate Boyer-Moore string matching. *SIAM Journal on Computing*. Vol. 22, No. 2, pp.243-260.
- [12] Ukkonen, E. 1985. Finding approximate patterns in strings. *Journal of Algorithms*. Vol. 6, No. 1, pp. 132-137.
- [13] Wagner, R and Fisher, M. 1974. The string-to-string correction problem. *Journal of the ACM*. Vol. 21, No. 1, pp. 168-173.